Predicting CYP Inhibition with Molecular Foundation Models

DTU Deep Learning Course

Supervisors: Kasper Helverskov Petersen and Jingchao Zhou

1 Description

Predictive AI is increasingly being used in drug discovery to help researchers screen molecules quickly and prioritize which ones to test experimentally. Molecular foundation models are particularly interesting in this context: they learn molecular representations from large datasets that can be fine-tuned to predict properties for which experimental data are limited. However, their performance depends on how they are adapted and trained. Exploring these choices gives you practical experience with a central task in applying AI to drug discovery.

In this project, you will explore one such application: predicting CYP inhibition. Cytochrome P450 (CYP) enzymes play an important role in drug metabolism. A molecule that inhibits these enzymes can alter how other drugs are metabolized, making CYP inhibition an important property to consider when developing new drugs.

The main model in this project is **Atom-JEPA** [1], a self-supervised model for learning representations of 3D atomistic structures. Its name stands for Joint-Embedding Predictive Architecture: during pretraining, it learns to predict latent representations of atoms and substructures from structural context. The resulting pretrained molecular representations can then be fine-tuned for property prediction, including CYP inhibition.

You will:

- implement and fine-tune molecular foundation models to predict CYP inhibition;
- compare their performance with a simpler baseline model using molecular fingerprints and/or descriptors;
- investigate how molecular representations and finetuning choices affect downstream prediction performance.

You will use two datasets that we source from the OpenADMET CYP Inhibition Blind Challenge to address two prediction tasks:

- **Regression:** predict four numerical targets for each test molecule, measuring how strongly it inhibits CYP1A2, CYP2C9, CYP2D6, and CYP3A4. The targets use a scale called pIC50: higher values mean less of the molecule is needed to reduce the enzyme's activity by half, indicating stronger inhibition.
- **Classification:** predict two binary (true/false) targets for each test molecule, indicating time-dependent inhibition of CYP2D6 and CYP3A4. In these experiments, researchers test whether allowing the molecule to interact with the enzyme before measuring its activity leads to stronger inhibition.

We provide starter code that downloads the data, creates training, validation, and test splits, and includes a basic working finetuning and evaluation pipeline for the pretrained Atom-JEPA model and four molecular-feature baselines.

To prepare inputs for 3D models (such as Atom-JEPA), the code also generates molecular *conformers* for each molecule in the datasets: possible three-dimensional arrangements of a molecule's atoms. These provide the atomic coordinates the model needs. Models using molecular strings or 2D graphs do not require this step.

You can also submit predictions to the project leaderboard to compare results with other groups. This is an optional addition to the project for fun and inspiration: participation and ranking do not affect your grade. Select your models using validation data, not leaderboard scores.

2 Project Tasks

Complete Parts 1–5 and document your methods, experiments, and results in your report. Part 6 is an optional extension and is not required to complete. During Parts 2–4, use validation performance to investigate training choices and select a final configuration for each prediction task. Present the selected models' test results together in **one final comparison table in Part 5**; separate test-result tables for Parts 2–4 are not required. You may use different training and fine-tuning protocols for the baseline, Atom-JEPA, and your additional foundation model.

1. **Set up the data and starter pipeline.**

   Prepare the data and generate molecular conformers using the starter code. Familiarize yourself with the two prediction tasks, the available labels, and the evaluation metrics. Keep the provided training/validation/test splits with the default split seed of 42 throughout your experiments.

2. **Explore a baseline model.**

   Choose one of the four molecular-feature baselines:

   (i) an MLP using Morgan fingerprints;
   (ii) an MLP using RDKit descriptors;
   (iii) an MLP using concatenated Morgan fingerprints and RDKit descriptors;
   (iv) LightGBM using concatenated Morgan fingerprints and RDKit descriptors.

   Investigate how training choices affect its validation performance on both prediction tasks. For the MLP baselines, you could explore network architecture, learning rate, batch size, or regularization. For LightGBM, you could explore learning rate, number of boosting iterations, tree complexity, or regularization. Among the configurations you tested, select the one that performs best on the validation set, and explain which changes helped and what you learned. Retain these configurations for the final comparison in Part 5.

3. **Fine-tune Atom-JEPA.**

   Fine-tune Atom-JEPA on both prediction tasks and investigate how to adapt its pretrained representations effectively. Use validation results to assess your experiments and compare them with your chosen baseline. Select a final configuration for each prediction task for the comparison in Part 5.

   Possible directions include:

- **Hyperparameter optimization:** explore learning rates, schedules, batch sizes, or other training hyperparameters. You may tune these manually or use a framework such as Optuna [2]. Always use validation performance to guide your choices.
- **Readout architecture:** design and evaluate an alternative to the readout architecture provided in the starter code.
- **Freezing strategies:** compare frozen features with full fine-tuning, or initially freeze the pretrained backbone and train only the readout, then unfreeze the backbone and train both together for the remaining steps. (Initially freezing the pretrained backbone is a very common choice when finetuning foundation models).
- **Regularization:** explore dropout, weight decay, or other regularization methods such as L2-SP.
- **Multiple conformers:** sample different conformers during training or average predictions over, for example, ten conformers per molecule at evaluation time.
- **Additional training data:** gather relevant external data and integrate it into the pipeline. Document the sources and how you integrate molecular structures, assay definitions, units, and labels. Remove overlaps with the provided validation and test molecules, keep the evaluation splits unchanged, and compare training with and without the additional data. For example:
  - Explore the CYP3A4 time-dependent inhibition surrogate dataset published by Novartis, which contains model-predicted labels for approximately 16,000 publicly available molecular structures [3].
  - This OpenADMET blog provides a discussion of using surrogate data for ADMET modeling, including its potential benefits and task-dependent limitations [4].

Investigate at least two distinct directions, either from the suggestions above or through other relevant ideas of your own. Your work will be assessed on the rationale for your choices, the depth of your investigation, and the quality of your experimental comparisons and analysis. Combining two routine changes, such as enabling weight decay and freezing the backbone, is not sufficient on its own to demonstrate a thorough investigation. Motivate your choices, use controlled comparisons to assess their effects, and explain what your results reveal about adapting Atom-JEPA to the two prediction tasks.

4. Implement and fine-tune another foundation model.

Choose a second pretrained molecular foundation model and integrate it into the pipeline. Explain your choice and how its inputs, representations, and pretraining differ from Atom-JEPA.

Check its input requirements, checkpoint availability, software dependencies, and computational cost before committing to it. Confirm that the chosen checkpoint can be fine-tuned and used with the inputs available for the project's test molecules. Report which pretrained checkpoint you use.

Fine-tune your chosen model on both prediction tasks. Build on your earlier experiments, but adapt the fine-tuning protocol to suit the model. Use validation results to select a final configuration for each prediction task for the comparison in Part 5.

Suggested options include the following; you may also choose another suitable pretrained molecular model:

1. **3D interatomic potentials:** AllScAIP [5] or PET-MAD [6], pretrained on atomistic energies and forces. Adapt their learned representations with a readout head.

2. **2D graph models:** Contrastive KERMT (CKERMT) [7], CheMeleon [8], MolCLR [9], or GROVER [10]. These use atom–bond connectivity and chemical features without requiring 3D conformers.
3. **Other models using 3D structure:** Monroe [11], which combines molecular graphs with 3D geometry, or Uni-Mol [12] and Uni-Mol2 [13], which learn representations from molecular conformers. Consider using the smaller model checkpoints to limit computational cost.
4. **Multimodal models and frameworks:** Mol-JEPA [14], pretrained using molecular and biochemical information, or MolProphecy [15], which combines molecular representations with LLM-generated chemical insights.
5. **SMILES models:** ChemBERTa-2 [16] or MoLFormer [17], which use molecular strings.

Depending on the model, you may extract embeddings from a frozen pretrained model and train a readout head, such as an MLP, on top (for example, with Mol-JEPA and Monroe), or fine-tune the model end-to-end. Choose and justify an approach suited to your selected model.

5. Complete the final comparison.

Evaluate the final validation-selected configurations on the test sets for **both regression and classification**. Present **one consolidated comparison table** with rows for:

(1) your chosen molecular-feature baseline;

(2) Atom-JEPA;

(3) your additional foundation model.

Use grouped columns or two panels within the same table for regression and classification. Report the aggregate project metrics: MA-ST-RAE for regression (lower is better) and mean MCC for classification (higher is better). Use the supplied evaluation code, which converts classification probabilities to true/false predictions at a threshold of 0.5 for MCC.

You may choose different protocols for different models and prediction tasks; describe and justify these differences. Discuss what the results tell you about molecular pretraining and your training choices. You are encouraged to aim for improvements over the reference results on the leaderboard, but outperforming them is not required. You may optionally submit your selected models' test predictions to the leaderboard as well!

6. Optional extra: explore an ensemble approach (not a required task).

Explore whether combining predictions from multiple models improves validation performance. For example, average predictions from your chosen baseline, Atom-JEPA, and the additional foundation model, or investigate a weighted ensemble. Select ensemble members and any combination weights using validation data only.

Throughout the project, fit models and preprocessing using training data, and choose configurations and checkpoints using validation data. Report test performance for the selected models, but **do not** use test results or leaderboard feedback to guide subsequent tuning. Finalize any optional ensemble before inspecting its test performance.

3 Materials

- **Starter code:** https://github.com/khelverskovp/dtu-cyp-project

- **Project leaderboard:** https://khelverskovp.github.io/dtu-cyp-leaderboard/
- **Atom-JEPA code and pretrained checkpoints:** see Reference [1].

The starter repository provides:

- data downloading and preparation, including 3D conformer generation and an approximately 70/15/15 training/validation/test split. The leaderboard uses this same test set;
- a basic working setup for fine-tuning Atom-JEPA, with a shared training and evaluation loop;
- baselines using Morgan fingerprints, RDKit descriptors, and their concatenation, as well as the lightgbm baseline;
- a modular structure for adding your own readout heads and additional foundation models;
- example job scripts for running experiments on the DTU HPC.

See the repository README for installation and execution instructions, and guidance on adding models and readout heads.

References

[1] K. Helverskov Petersen et al. *Atom-JEPA: Joint-Embedding Predictive Architecture for 3D Atomistic Systems*. arXiv:2610.08400, 2026. https://arxiv.org/abs/2610.08400. Code and checkpoints: https://github.com/khelverskovp/atom-jepa.

[2] T. Akiba et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. KDD, 2019. https://arxiv.org/abs/1907.10902.

[3] A. Fluetsch et al. *Deep Learning Models Compared to Experimental Variability for the Prediction of CYP3A4 Time-Dependent Inhibition*. Chemical Research in Toxicology, 37(4):549–560, 2024. https://doi.org/10.1021/acs.chemrestox.3c00305.

[4] M. Castellanos. *Not invited to the party? How to make proprietary ADMET insights more accessible*. OpenADMET blog, n.d. https://openadmet.ghost.io/not-invited-to-the-party-how-to-make-proprietary-admet-insights-more-accessible/.

[5] *A recipe for scalable attention-based MLIPs: unlocking long-range accuracy with all-to-all node attention*. arXiv:2603.06567, 2026. https://arxiv.org/abs/2603.06567.

[6] *PET-MAD as a lightweight universal interatomic potential for advanced materials modeling*. Nature Communications, 2025. https://doi.org/10.1038/s41467-025-65662-7.

[7] Y. Xue et al. *Probabilistic Contrastive Pretraining for Multi-task ADME Property Prediction*. arXiv:2606.11508v1, 2026. https://arxiv.org/abs/2606.11508v1. Code: https://github.com/NVIDIA-BioNeMo/KERMT.

[8] J. W. Burns et al. *Deep Learning Foundation Models from Classical Molecular Descriptors*. arXiv:2506.15792, 2025 (revised 2026). https://arxiv.org/abs/2506.15792. Fine-tuning guide: https://chemprop.readthedocs.io/en/latest/chemeleon_foundation_finetuning.html.

[9] Y. Wang et al. *Molecular contrastive learning of representations via graph neural networks*. Nature Machine Intelligence, 2022. https://doi.org/10.1038/s42256-022-00447-x. Code: https://github.com/yuyangw/MolCLR.

[10] Y. Rong et al. *Self-Supervised Graph Transformer on Large-Scale Molecular Data*. NeurIPS, 2020. Code: https://github.com/tencent-ailab/grover.

[11] B. Banaszewski and A. W. Fitzgibbon. *Monroe: A Molecular Foundation Model for In-Context Probabilistic Inference*. arXiv:2608.18982, 2026. https://arxiv.org/abs/2608.18982. Code: https://github.com/blazejba/monroe.

[12] G. Zhou et al. *Uni-Mol: A Universal 3D Molecular Representation Learning Framework*. ICLR, 2023. Code: https://github.com/deepmodeling/Uni-Mol.

[13] X. Ji et al. *Uni-Mol2: Exploring Molecular Pretraining Model at Scale*. arXiv:2406.14969, 2024. https://arxiv.org/abs/2406.14969.

[14] F. Rottach et al. *Mol-JEPA: A multimodal Joint Embedding Predictive Architecture for Molecules*. arXiv:2608.22642, 2026. https://arxiv.org/abs/2608.22642. Checkpoint: https://huggingface.co/Flogrammer/Mol-JEPA.

[15] J. Zhao et al. *MolProphecy: Bridging Medicinal Chemists' Knowledge and Molecular Pre-Trained Models via a Multi-Modal Framework*. arXiv:2507.02932v1, 2025. https://arxiv.org/abs/2507.02932v1. Code: https://github.com/zhangruochi/MolProphecy.

[16] W. Ahmad et al. *ChemBERTa-2: Towards Chemical Foundation Models*. arXiv:2209.01712, 2022. https://arxiv.org/abs/2209.01712. Example checkpoint: https://huggingface.co/DeepChem/ChemBERTa-77M-MLM.

[17] J. Ross et al. *Large-scale chemical language representations capture molecular structure and properties*. Nature Machine Intelligence, 2022. https://doi.org/10.1038/s42256-022-00580-7. Code: https://github.com/IBM/molformer.